

AI-Powered Pharmaceutical Complaint Management System

1. Introduction

The AI-Powered Pharmaceutical Complaint Management System is an intelligent complaint intake and assessment application designed to simplify the process of recording, analyzing, and managing pharmaceutical customer complaints.

The system provides an AI Copilot through which users can enter complaint information as natural-language text or upload complaint documents such as PDFs. The application extracts relevant complaint information and automatically populates a structured complaint form.

The system also uses a LangGraph-based workflow to determine whether the complaint information is complete, request missing information when necessary, support editing of existing information, and perform AI-based complaint assessment.

Once sufficient information is available, the AI determines the complaint category, evaluates risk severity, provides a reason for the risk level, suggests the next action, identifies possible root-cause hypotheses, and generates corrective and preventive actions (CAPA).

The completed complaint can then be committed to the QMS through the backend.

2. Project Objective

The primary objectives of the project are:

- Automate pharmaceutical complaint data extraction.
 - Allow users to enter complaints using natural language.
 - Support complaint information extraction from uploaded documents.
 - Reduce manual data entry.
 - Automatically identify missing complaint information.
 - Allow users to update or edit extracted complaint information.
 - Automatically classify complaint categories.
 - Perform AI-based risk assessment.
 - Generate possible root-cause hypotheses.
 - Generate corrective and preventive actions.
 - Provide a user-friendly AI Copilot interface.
 - Allow completed complaints to be committed to the QMS.
-

3. Key Features

3.1 Natural Language Complaint Input

Users can describe complaints directly in the AI Copilot.

For example:

Customer ABC reported that Paracetamol 500 mg from batch BTH24053 has damaged packaging. The complaint was received by email on August 8, 2026.

The backend processes this information and extracts structured fields.

3.2 PDF/File Upload

The frontend allows users to upload complaint documents.

Supported frontend file types include:

- PDF
- Images
- TXT
- EML

The uploaded file is sent to the backend using a multipart form request.

The backend processes the document and uses the extracted text as complaint input for the AI workflow.

3.3 AI Complaint Extraction

The extraction workflow identifies:

- Complaint Source
- Customer Name
- Product Name
- Product Strength
- Batch Number
- Manufacturing Date
- Expiry Date
- Quantity Affected
- Complaint Date
- Complaint Description

The AI is instructed to extract only information explicitly available in the input and not guess missing information.

3.4 Date Normalization

The backend normalizes extracted dates into:

YYYY-MM-DD

For example:

August 8 2026

becomes:

2026-08-08

The workflow separately handles:

- Manufacturing date
- Expiry date
- Complaint date

This prevents confusion between different types of dates.

3.5 Completeness Checking

After extraction, the system checks whether all required complaint fields are available.

Required fields include:

1. Complaint Source
2. Customer Name
3. Product Name
4. Product Strength
5. Batch Number
6. Manufacturing Date
7. Expiry Date
8. Quantity Affected
9. Complaint Date
10. Detailed Complaint Description

If information is missing, the system does not proceed directly to AI assessment.

Instead, it displays the missing information to the user.

3.6 Complaint Editing

The user can modify existing complaint information using natural language.

For example:

Change the batch number to BTH24054.

The system identifies this as an edit request and updates only the requested field.

The remaining complaint information is preserved.

3.7 AI Complaint Categorization

Once the complaint contains sufficient factual information, the AI determines the appropriate complaint category.

Possible categories include:

- Product Quality
- Packaging
- Labeling
- Delivery
- Product Availability
- Adverse Event
- Manufacturing
- Storage
- Other

The category is determined automatically based on the actual complaint.

3.8 Risk Assessment

The AI evaluates the complaint and assigns one of four risk levels:

- Low
- Medium
- High
- Critical

The system also provides a reason explaining the selected risk level.

3.9 Suggested Next Action

The AI provides a recommended next investigation or action based on the complaint.

This helps the user understand what should happen after the complaint has been received.

3.10 Possible Root Causes

The AI generates possible root causes as hypotheses.

The system explicitly treats these as hypotheses rather than confirmed investigation results.

This distinction is important because the AI does not have access to actual laboratory or investigation results.

3.11 CAPA Generation

The system generates:

Corrective Action

An action intended to address the current complaint and investigate the issue.

Preventive Action

An action intended to reduce the possibility of recurrence.

Both are required in the assessment output.

3.12 QMS Commit

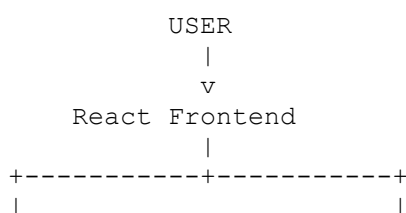
After all required information is available, the user can commit the complaint to the QMS.

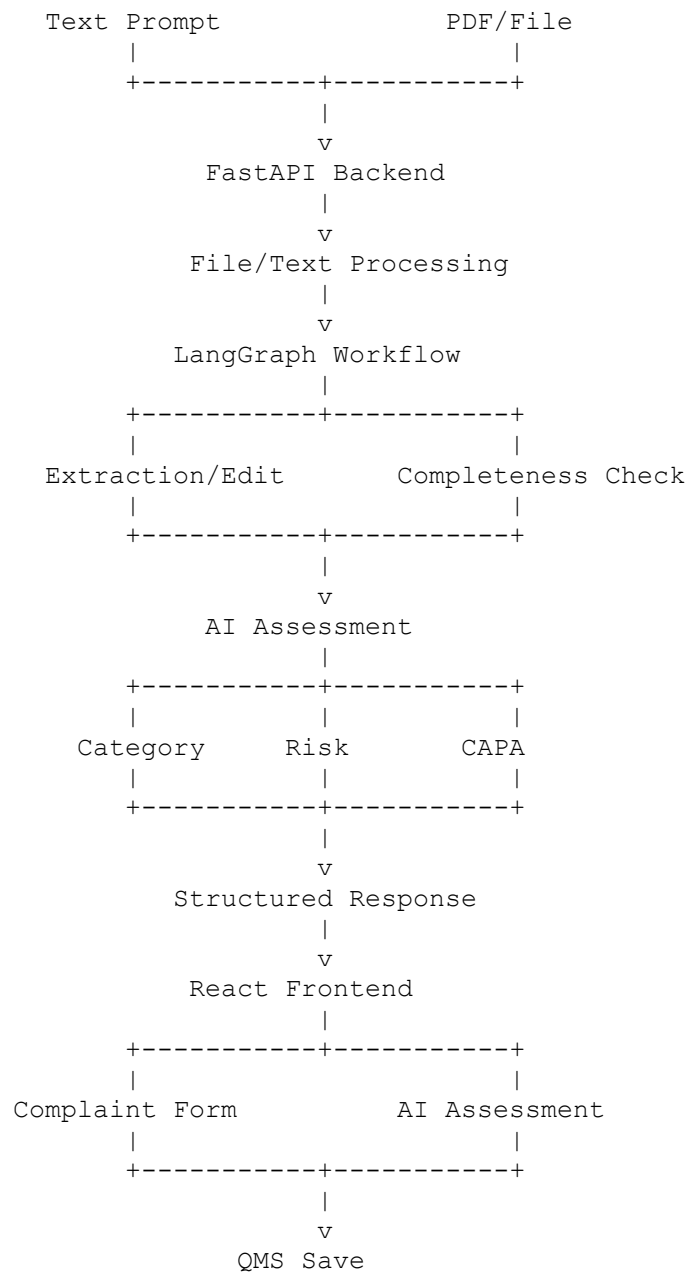
The frontend sends the structured complaint information to the backend `/ai/save` endpoint.

The backend returns a complaint ID after successful storage.

4. System Architecture

The system follows a frontend-backend-AI workflow.





5. Technology Stack

Frontend

- React
- JavaScript
- CSS
- Fetch API

Backend

- Python
- FastAPI

AI / LLM

- LangChain
- LangGraph
- ChatGroq
- Llama 3.3 70B Versatile

Data Validation

- Pydantic

Environment Configuration

- python-dotenv

Document Processing

The backend receives uploaded files and extracts their textual content before sending the complaint information into the AI workflow.

6. Frontend Implementation

The main frontend component is:

`App.jsx`

The frontend maintains the complaint information using React state.

The initial complaint structure contains fields such as:

```
complaint_source
customer_name
product_name
product_strength
batch_number
manufacturing_date
expiry_date
quantity_affected
complaint_category
complaint_date
description
risk_severity
risk_reason
suggested_next_action
possible_root_causes
corrective_action
preventive_action
```

6.1 State Management

The application uses React's `useState`.

Important states include:

```
data
copilotMessage
chatHistory
selectedFile
loading
message
completeness
savedComplaintId
```

The `data` state contains the current complaint information.

The `selectedFile` state stores the uploaded document.

The `chatHistory` state stores the conversation displayed in the AI Copilot.

6.2 Updating Complaint Fields

The frontend contains an `updateField()` function.

Its purpose is to update individual complaint fields.

Conceptually:

```
User edits field
      |
      v
updateField()
      |
      v
React state updated
      |
      v
Form displays new value
```

6.3 Updating the Form from AI

The `updateFromAI()` function receives the backend response and maps the AI-generated fields into the React state.

For example:

```
AI response
      |
      v
updateFromAI()
```



```
      v
setData()
      |
      v
Complaint form updated
```

This is what allows extracted information to automatically appear in the right-side complaint form.

6.4 Sending User Requests

The main frontend processing function is:

```
sendCopilotMessage()
```

It handles both:

- Text requests
- File uploads

If a file exists, the frontend creates a `FormData` object.

The file is added using:

```
formData.append("file", selectedFile)
```

The user's message and current complaint data are also sent.

6.5 Text Request API

For normal text input, the frontend sends a JSON request to:

```
POST /ai/copilot
```

The request contains:

```
message
current_data
```

The backend then processes the complaint using the LangGraph workflow.

6.6 File Upload API

For uploaded files, the frontend sends the request to:

POST /ai/copilot/file

The request uses multipart/form-data.

The uploaded document is sent as:

file

The user's optional message is also included.

7. Backend Processing

The backend acts as the bridge between the React frontend and AI workflow.

The basic workflow is:

```
Frontend
  |
  v
FastAPI endpoint
  |
  v
Read request
  |
  v
Extract text if file uploaded
  |
  v
Create complaint state
  |
  v
Invoke LangGraph
  |
  v
Receive structured result
  |
  v
Return JSON response
```

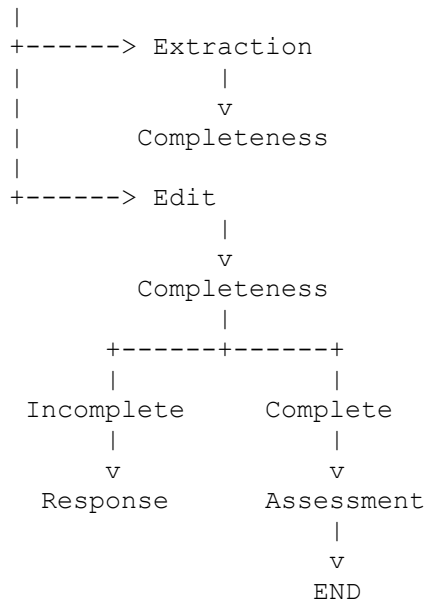
8. LangGraph Workflow

The main workflow is implemented in:

graph.py

The LangGraph workflow contains the following major nodes:

```
START
  |
  v
Routing
```



9. Complaint State

The `ComplaintState` class is the shared state passed between `LangGraph` nodes.

It contains:

- Complaint input
- Customer information
- Product information
- Dates
- Complaint description
- Risk assessment
- Root causes
- CAPA
- Missing fields
- Workflow stage
- Response message

The state allows different nodes to work on the same complaint progressively.

10. Structured Output Models

The workflow uses Pydantic models to ensure structured AI responses.

ComplaintExtraction

Stores extracted complaint information.

ComplaintEdit

Stores updated complaint information.

CAPA

Contains:

```
corrective_action  
preventive_action
```

ComplaintAssessment

Contains:

```
complaint_category  
severity_of_risk  
risk_reason  
suggested_next_action  
possible_root_causes  
capa
```

This reduces the chance of receiving unstructured AI output.

11. LLM Configuration

The project uses:

```
ChatGroq
```

with:

```
llama-3.3-70b-versatile
```

The temperature is set to:

```
0
```

This is useful for structured extraction because deterministic behavior is preferred.

The API key is loaded using environment variables:

```
GROQ_API_KEY
```

12. Extraction Node

The function:

`extraction_node()`

takes the complaint text and sends it to the structured extraction model.

The prompt specifically instructs the model:

- Extract only explicitly available information.
- Do not guess missing information.
- Keep complaint, manufacturing, and expiry dates separate.
- Do not determine risk at this stage.
- Do not generate CAPA at this stage.

The extracted information is then merged into the existing state.

13. Edit Node

The function:

`edit_node()`

handles natural-language modifications.

For example:

Change quantity affected to 25.

The model receives the existing complaint data and the user's request.

It is instructed to:

- Change only what the user requested.
- Preserve all other fields.
- Never invent information.
- Return the complete updated complaint.

The updated information is then merged into the state.

14. Completeness Node

The function:

`completeness_node()`

checks all required fields.

If fields are missing:

```
completeness_status = "Incomplete"
```

The workflow generates a clarification message.

For example:

```
Please provide:  
Batch Number, Expiry Date, Complaint Date.
```

If all fields are available:

```
completeness_status = "Complete"
```

The workflow proceeds to AI assessment.

15. Assessment Node

The function:

```
assessment_node()
```

is the main AI reasoning stage.

It determines:

Complaint Category

Example:

```
Packaging
```

Risk Severity

One of:

```
Low  
Medium  
High  
Critical
```

Risk Reason

Explains why the risk level was selected.

Suggested Next Action

Provides the recommended investigation/action.

Possible Root Causes

Provides possible causes as hypotheses.

CAPA

Generates:

```
Corrective Action
Preventive Action
```

The result is returned as structured data.

16. Routing Logic

The workflow contains two important routing functions.

route_from_start()

Determines whether the user's request is:

```
Extraction
```

or:

```
Edit
```

If there is no existing complaint information, the system performs extraction.

If existing information is present, the system uses an LLM classification step to determine whether the latest message is an edit or additional information.

route_after_change()

After extraction or editing, this function checks:

```
Incomplete?
```

If yes:

```
incomplete
```

Otherwise:

```
assessment
```

17. End-to-End Example

Consider the following user input:

Customer ABC complained that Paracetamol 500 mg,
batch BTH24053, has damaged packaging.
The complaint was received by email on August 8 2026.

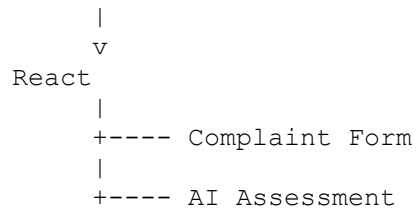
The workflow is:

```
User Input
  |
  v
React AI Copilot
  |
  v
POST /ai/copilot
  |
  v
FastAPI
  |
  v
LangGraph
  |
  v
Extraction Node
  |
  v
Extract fields
  |
  v
Completeness Check
```

If required fields are missing, the system asks the user to provide them.

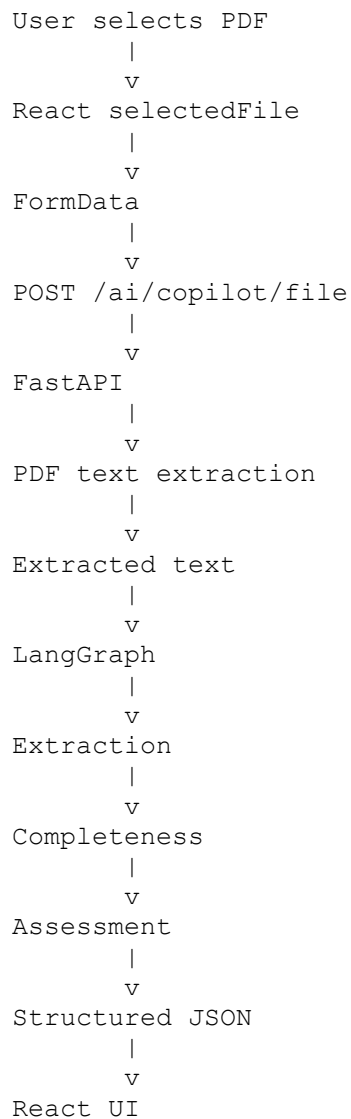
Once complete:

```
Completeness
  |
  v
Assessment Node
  |
  +---- Complaint Category
  |
  +---- Risk Severity
  |
  +---- Risk Reason
  |
  +---- Next Action
  |
  +---- Root Causes
  |
  +---- Corrective Action
  |
  +---- Preventive Action
  |
  v
JSON Response
```

18. PDF Workflow

For a PDF complaint:



The important point is that the PDF itself is not directly passed to the LLM as complaint information. The backend first extracts its text and then uses that text as the input to the complaint workflow.

19. QMS Save Workflow

Once the complaint is complete:

```
Commit to QMS
  |
  v
POST /ai/save
  |
  v
FastAPI
  |
  v
Validate complaint
  |
  v
Store complaint
  |
  v
Return Complaint ID
  |
  v
React
  |
  v
"Complaint committed to QMS"
```

The frontend displays the returned complaint ID.

20. Error Handling

The frontend handles unsuccessful API responses.

It checks:

```
response.ok
```

If the request fails, the backend error is displayed to the user.

The UI also maintains a loading state:

```
Processing...
```

or:

```
Committing...
```

This prevents users from repeatedly submitting requests while processing is underway.

21. UI Design

The application has a two-panel interface.

Left Panel

Contains:

- AI Complaint Copilot
- Chat history
- Text input
- File upload
- Missing information messages
- AI interaction

Right Panel

Contains:

- Complaint information
- Customer details
- Product details
- Complaint details
- AI assessment
- Risk information
- Root causes
- CAPA
- QMS commit button

The right side is designed to allow the user to review the structured complaint before committing it.

22. Benefits

The system provides several benefits:

- Reduces manual complaint entry.
- Supports natural-language interaction.
- Automates complaint information extraction.
- Supports document-based complaint intake.
- Detects missing information.
- Allows natural-language editing.
- Provides consistent AI-based risk assessment.
- Automatically generates CAPA suggestions.
- Provides human review before QMS commitment.
- Separates factual extraction from AI assessment.

23. Limitations

The AI-generated assessment is intended to support the complaint management process and should not replace formal quality investigation.

Possible root causes are hypotheses and must be verified through appropriate investigation.

AI-generated CAPA recommendations should also be reviewed by qualified personnel before implementation.

The system depends on the quality and completeness of the input complaint information.

24. Future Enhancements

Possible future improvements include:

- OCR for scanned PDFs and images.
 - Email integration.
 - Database dashboards.
 - Complaint search and filtering.
 - User authentication and role-based access.
 - Audit trails.
 - Complaint status tracking.
 - Automated email notifications.
 - Advanced analytics.
 - Integration with enterprise QMS systems.
 - Human approval workflow.
 - More advanced document classification.
-

25. Conclusion

The AI-Powered Pharmaceutical Complaint Management System combines a React frontend, FastAPI backend, structured LLM processing, and LangGraph workflow orchestration to automate pharmaceutical complaint intake and assessment.

The system accepts natural-language complaints and uploaded documents, extracts structured information, checks completeness, supports editing, and performs AI-based complaint categorization, risk assessment, root-cause hypothesis generation, and CAPA recommendation.

The final structured information is presented to the user for review before being committed to the QMS.

The project demonstrates how generative AI and workflow orchestration can be applied to automate a structured quality-management process while retaining human review and control.